GitHub Token Generation, Permissions, and Release Monitoring Guide

Overview

This document explains:

1. How to generate a GitHub token using a bot account
2. How GitHub token permissions work
3. Why GitHub APIs use Bearer tokens
4. How GitHub API URLs are queried for daily, weekly, and monthly trending repositories
5. Manual API test runs for:
    - PowerShell (Windows)
    - macOS
    - Linux

---

1. Generating a GitHub Token Using a Bot Account

Why Use a Bot Account?

Instead of using a personal GitHub account for automation, it is recommended to create a separate bot account.

Example bot account names:

```
ai-tech-radar-bot
opensource-monitor-bot
github-release-monitor-bot
```

Benefits:

- Keeps automation separate from personal work
- Easier to manage permissions
- Safer for production systems
- Easier to revoke access if compromised

Step-by-Step Token Generation

Step 1 — Login to GitHub Bot Account

Open:

```
https://github.com
```

Login using your bot account.

Step 2 — Open Developer Settings

Navigate to:

```
Settings
→ Developer Settings
→ Personal Access Tokens
```

Direct URL:

```
https://github.com/settings/tokens
```

Step 3 — Generate New Token

Choose:

```
Fine-grained personal access token
```

GitHub recommends fine-grained tokens because they are:

- More secure
- Repository scoped
- Permission scoped
- Easier to control

Step 4 — Enter Token Details

Example Token Name

```
github-release-monitor-bot
```

Example Description

```
Token used for testing GitHub webhook integrations, release monitoring, API r
equests, AI-generated summaries, and automated team notifications.
```

2. Understanding GitHub Token Permissions

What Are Permissions?

Developer Docs: https://docs.github.com/en/rest/authentication/permissions-required-for-fine-grained-personal-access-tokens?apiVersion=2026-03-10

Permissions define:

```
What your token is allowed to access
```

GitHub follows a security principle called:

```
Least Privilege Access
```

Meaning:

```
Only give the minimum permissions required
```

Recommended Repository Access

For public release monitoring systems:

Select:

```
Public repositories only
```

Recommended Permissions

Permission	Recommended Access	Purpose
Metadata	Read-only	Repository info, stars, releases
Contents	Read-only	README, changelog, release notes
Issues	Read-only (optional)	Track issue updates
Pull Requests	Read-only (optional)	Analyze PRs

Permissions Explanation

Metadata — Read-only

Allows access to:

- Repository name
- Stars
- Repository description
- Release metadata
- Public repository information

This is the most important permission.

Contents — Read-only

Allows access to:

- README files
- Release notes
- Changelogs
- Repository files

This is required for AI summarization systems.

Issues — Read-only

Optional permission.

Useful for:

- Bug tracking
- Feature tracking
- AI summaries of issues

Pull Requests — Read-only

Optional permission.

Useful for:

- Monitoring PR updates, AI analysis of development activity

Permissions You Should Avoid Initially

Avoid enabling:

```
Administration
Write access
Delete access
Secrets
Actions write
```

These are unnecessary for monitoring systems and increase security risks.

---

3. Understanding Bearer Tokens

Important Concept

GitHub APIs use:

```
Bearer Token Authentication
```

NOT:

```
Authorization-only raw token strings
```

Correct Format

```
Authorization: Bearer YOUR_TOKEN
```

Example:

```
Authorization: Bearer github_pat_xxxxxxxxx
```

Incorrect Format

```
Authorization: github_pat_xxxxxxxxx
```

This is invalid because GitHub needs to know the authentication type.

Difference Between Authorization and Bearer

Authorization

This is the HTTP header name.

Example:

```
Authorization: ...
```

Bearer

This is the authentication scheme.

Meaning:

```
Whoever possesses this token can access the API
```

Structure

```
Authorization: <TYPE> <TOKEN>
```

Example:

```
Authorization: Bearer github_pat_xxxxx
```

---

4. GitHub Trending Repository Queries

Important Note

GitHub does NOT officially provide a trending API.

Most systems use:

- GitHub Search API
- Star growth calculations
- Repository creation dates

Daily Trending Repositories

This query finds repositories created today and sorts them by stars.

Query Logic

```
created after today's date
sorted by stars descending
```

API URL

```
https://api.github.com/search/repositories?q=created:>2026-05-05&sort=stars&order=desc&per_page=10
```

Weekly Trending Repositories

API URL

```
https://api.github.com/search/repositories?q=created:>2026-04-29&sort=stars&order=desc&per_page=10
```

Monthly Trending Repositories

API URL

```
https://api.github.com/search/repositories?q=created:>2026-04-01&sort=stars&order=desc&per_page=10
```

Understanding Query Parameters

Parameter	Meaning
created:>DATE	Repositories created after the specified date
sort=stars	Sort by star count
order=desc	Highest stars first
per_page=10	Return top 10 repositories

---

5. Release Monitoring Queries

Latest Release of LangChain

Repository:

`langchain-ai/langchain`

Latest Release URL

`https://api.github.com/repos/langchain-ai/langchain/releases/latest`

All Releases URL

`https://api.github.com/repos/langchain-ai/langchain/releases`

Detecting Major Releases

Major releases usually follow:

```
v1.0.0
v2.0.0
v3.0.0
```

These are considered major updates.

---

6. Manual API Test Runs

A. PowerShell (Windows)

<u>Create Headers</u>

```
$headers = @{
    Authorization = "Bearer YOUR_TOKEN"
}
```

Test User Authentication

```
Invoke-RestMethod `
    -Uri "https://api.github.com/user" `
    -Headers $headers
```

Test Latest LangChain Release

```
Invoke-RestMethod `
    -Uri "https://api.github.com/repos/langchain-ai/langchain/releases/latest
" `
    -Headers $headers
```

Daily Trending Repositories

```
Invoke-RestMethod `
    -Uri "https://api.github.com/search/repositories?q=created:>2026-05-05&so
rt=stars&order=desc&per_page=10" `
    -Headers $headers
```

Weekly Trending Repositories

```
Invoke-RestMethod `
    -Uri "https://api.github.com/search/repositories?q=created:>2026-04-29&so
rt=stars&order=desc&per_page=10" `
    -Headers $headers
```

Monthly Trending Repositories

```
Invoke-RestMethod `
    -Uri "https://api.github.com/search/repositories?q=created:>2026-04-01&so
rt=stars&order=desc&per_page=10" `
    -Headers $headers
```

Releases Published Today

```
$releases = Invoke-RestMethod `
    -Uri "https://api.github.com/repos/langchain-ai/langchain/releases" `
    -Headers $headers
$today = (Get-Date).ToString("yyyy-MM-dd")
$releases | Where-Object {
    $_.published_at -like "$today*"
}
```

B. macOS and Linux

Test User Authentication

```
curl -H "Authorization: Bearer YOUR_TOKEN" \
https://api.github.com/user
```

Latest LangChain Release

```
curl -H "Authorization: Bearer YOUR_TOKEN" \
https://api.github.com/repos/langchain-ai/langchain/releases/latest
```

Daily Trending Repositories

```
curl -H "Authorization: Bearer YOUR_TOKEN" \
"https://api.github.com/search/repositories?q=created:>2026-05-05&sort=stars&
order=desc&per_page=10"
```

Weekly Trending Repositories

```
curl -H "Authorization: Bearer YOUR_TOKEN" \
"https://api.github.com/search/repositories?q=created:>2026-04-29&sort=stars&
order=desc&per_page=10"
```

Monthly Trending Repositories

```
curl -H "Authorization: Bearer YOUR_TOKEN" \
"https://api.github.com/search/repositories?q=created:>2026-04-01&sort=stars&
order=desc&per_page=10"
```

---

7. Monitoring Repositories Without Starring Them

You do NOT need to:

- Follow repositories
- Star repositories
- Watch repositories

for GitHub API monitoring.

Your monitoring system can directly query any public repository using:

```
owner/repository
```

Example:

```
langchain-ai/langchain
ollama/ollama
openai/openai-python
```

Recommended Monitoring Strategy

Create a repository list:

```
[
  "langchain-ai/langchain",
  "ollama/ollama",
  "openai/openai-python"
]
```

Then:

1. Periodically query releases
2. Compare latest release tags
3. Detect changes
4. Generate AI summaries
5. Send notifications to team systems

Example Workflow

```
GitHub API
      ↓
Get Latest Release
      ↓
Extract Release Notes
      ↓
AI Summarization
      ↓
Watsapp / Mail Alerts
```

Security Recommendations

Never Expose Tokens Publicly

Do NOT:

- Push tokens to GitHub repositories
- Paste tokens into screenshots
- Share tokens in public chats

Store Tokens in Environment Variables

Example:

```
GITHUB_TOKEN=github_pat_xxxxxxxxx
```

---

8. Test Repository and Example Output Generation

Test Repository

Repository URL:

```
https://github.com/haroontrailblazer-CloseFuture/github-connectivity
```

Specific Commit:

```
https://github.com/haroontrailblazer-CloseFuture/github-connectivity/commit/ada962282269893b20f53e325d5edc148a02e7a1
```

Purpose of the Test Repository

This repository contains a JavaScript-based GitHub API testing workflow used for:

- GitHub API connectivity testing
- Release monitoring queries
- Trending repository analysis
- Report generation
- Token validation

The repository uses:

`GitHub Secrets and Variables` for securely storing the GitHub token.

Token Storage Recommendation

Instead of hardcoding tokens directly into JavaScript files:

```
const token = "github_pat_xxxxxxxxx";
```

use:

```
GitHub Repository Secrets
```

Example:

```
Settings
→ Secrets and Variables
→ Actions
→ New Repository Secret
```

Example secret name:

```
GITHUB_TOKEN
```

Example Header Replacement

Inside the JavaScript file, replace:

```
Authorization: "Bearer PASTE_YOUR_GITHUB_TOKEN"
```

with:

```
Authorization: `Bearer ${process.env.GITHUB_TOKEN}`
```

This allows the token to be securely injected from GitHub Secrets.

Expected Workflow

The JavaScript test file performs:

1. GitHub API requests
2. Trending repository queries
3. Release monitoring queries
4. Result aggregation
5. Report generation

Generated Outputs

The script generates:

JSON Report

`report.json`

Contains:

- Raw GitHub API response data
- Trending repository metadata
- Release metadata
- Query outputs

Markdown Report

`report.md`

Contains:

- Human-readable summaries
- Trending repository tables
- Release summaries
- Structured monitoring reports

Example Generated Report Structure

report.json

```
{
  "trending_today": [],
  "trending_week": [],
  "trending_month": [],
  "latest_releases": []
}
```

---

report.md

```
# GitHub Monitoring Report

## Trending Today
- Repository A
- Repository B

## Weekly Trending
- Repository X

## Latest Releases
- LangChain v1.0.0
```

Recommended Test Run Procedure

Step 1

Clone repository:

```
git clone https://github.com/haroontrailblazer-CloseFuture/github-connectivity.git
```

Step 2

Install dependencies:

```
npm install
```

Step 3

Create environment variable:

```
GITHUB_TOKEN=github_pat_xxxxxxxxx
```

Step 4

Run the test script:

```
node index.js
```

Expected Output

The script should:

1. Authenticate with GitHub API
2. Fetch trending repositories
3. Fetch release information
4. Generate:

```
report.json
report.md
```

Recommended Improvements

Future improvements may include:

- AI-generated summaries
- Discord notifications
- Telegram notifications
- Scheduled cron jobs
- Webhook integrations
- MCP migration
- Star-growth analytics
- Release importance ranking

Conclusion

This setup provides:

- GitHub release monitoring
- Trending repository tracking
- AI-generated summaries
- Team notifications
- Cross-platform API testing

without requiring MCP initially.